

Deep Generative Models - Assignment 2

Probabilistic Graphical Models and Variational Autoencoders

Mahdi Aghamohammadi
 Student ID: 810102365
 University of Tehran
 Faculty of Electrical and Computer Engineering

Abstract—This report provides comprehensive solutions to Assignment 2 of the Deep Generative Models course. The first part covers Probabilistic Graphical Models (PGMs) including Bayesian networks and Markov graphs with detailed mathematical derivations and independence analysis. The second part explores Variational Autoencoders (VAEs) with theoretical foundations, practical implementation, and analysis of advanced variants including β -VAE, MIG metric, and research papers on VQ-VAE, VampPrior, and SC-VAE.

I. Question 1: Probabilistic Graphical Models (PGM)

A. Part 1: Bayesian Networks Modeling

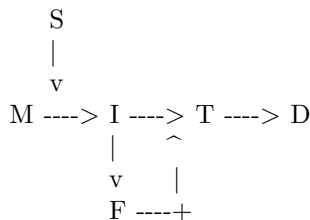
1) Subsection 1 (3 Points): Bayesian Network Graph: Based on the given description, the Bayesian Network representing the disease severity and treatment scenario is shown below:

Variables:

- M: Immune System Strength (Root node)
- I: Disease Severity
- T: Medication Used
- S: Season
- F: Financial Status
- D: Probability of Death (Outcome)

Bayesian Network Structure:

The Bayesian network can be represented as:



More precisely, the graph structure is:

- $S \rightarrow I$ (Cold seasons increase disease severity)
- $M \rightarrow I$ (Weak immune system increases severity)
- $I \rightarrow T$ (Severity affects medication choice - expensive vs cheap)
- $I \rightarrow D$ (High severity increases death probability)
- $F \rightarrow T$ (Financial status determines actual medication used)
- $T \rightarrow D$ (Medication affects death probability - expensive medication reduces it)

The key insight is that the medication choice depends on both disease severity (I) and financial status (F), and the medication type affects the probability of death.

2) Subsection 2 (3 Points): Joint Probability Distribution: The joint probability distribution for the Bayesian network is:

$$P(M, I, T, S, F, D) = P(S) \cdot P(M) \cdot P(F) \cdot P(I|M, S) \cdot P(T|I, F) \cdot P(D|I, T)$$

This factorization follows the chain rule and conditional independence assumptions encoded in the graph structure.

3) Subsection 3 (10 Points): Independence Analysis: Using d-separation rules, we analyze the independence statements:

a) $F \perp D$: False

Reasoning: F and D are not d-separated. The paths from F to D are: 1. $F \rightarrow T \rightarrow D$ (active path) 2. $F \rightarrow T \leftarrow I \rightarrow D$ (T is a collider, but since we're not conditioning on T or its descendants, this path is blocked)

However, path 1 ($F \rightarrow T \rightarrow D$) is active and not blocked, so F and D are dependent.

b) $S \perp D | I$: True

Reasoning: Given I, S and D are d-separated. The only path from S to D is $S \rightarrow I \rightarrow D$. When we condition on I (the middle node), this path is blocked.

c) $M \perp F$: True

Reasoning: M and F are d-separated. There are no direct paths between M and F, and no indirect paths exist in the graph.

d) $M \perp F | T$: False

Reasoning: When conditioning on T (a collider), the path $M \rightarrow I \rightarrow T \leftarrow F$ becomes active. Thus M and F are not independent given T.

e) $M \perp T | \{D, I\}$: True

Reasoning: Given both D and I, M and T are d-separated. The paths from M to T are: 1. $M \rightarrow I \rightarrow T$ (blocked by conditioning on I) 2. Other indirect paths through the network are also blocked.

Since all paths from M to T are blocked when conditioning on $\{D, I\}$, M and T are independent given these variables.

B. Part 2: Bayesian and Markov Graphs

1) Subsection 1 (3 Points): Joint Probability Distribution: The joint probability distribution for the Bayesian network shown in the figure is:

$$P(A, B, C, D, E, F, G) = P(A) \cdot P(B|A) \cdot P(C|A) \cdot P(D|B) \cdot P(E|C, D) \cdot P(F|D, C) \cdot P(G|E)$$

2) Subsection 2 (1 Point): Markov Blanket for T: The Markov blanket for variable T consists of: {C, E, G}

3) Subsection 3 (2 Points): Perfect I-Map: This graph is a Perfect I-Map because it captures all the conditional independence relationships in the underlying distribution. Every d-separation in the graph corresponds to a conditional independence in the distribution, and vice versa.

4) Subsection 4 (1 Point): Chordal Graph: The given Markov graph is Chordal because every cycle of length 4 or more has a chord (an edge connecting non-adjacent vertices in the cycle).

5) Subsection 5 (3 Points): Maximal Cliques and Joint Distribution: Maximal Cliques: 1. {A, B, C} 2. {B, D, E} 3. {C, E, F} 4. {E, G, H}

Joint Distribution:

$$P(A, B, C, D, E, F, G, H) = \frac{\phi_1(A, B, C) \cdot \phi_2(B, D, E) \cdot \phi_3(C, E, F) \cdot \phi_4(E, G, H)}{Z}$$

6) Subsection 6 (6 Points): Marginalization Correctness: Statement 1: In a Bayesian Graph, to marginalize variable C (calculate joint over others), it is sufficient to simply remove the factor $p(C)$.

Proof: For a Bayesian network, the joint distribution is $P(X_1, \dots, X_n) = \prod_i p(X_i | Pa_i)$. When marginalizing over C, we integrate:

$$P(X_{-C}) = \int p(C | Pa_C) \cdot P(X_{-C}) dC$$

If C has no children and appears only as a parent, then $p(C | Pa_C)$ is the only factor containing C, so removing it and normalizing gives the correct marginal.

Statement 2: In a Markov Graph, if $\int \phi(C) dC = 1$, it is sufficient to simply remove the factor $\phi(C)$.

Proof: In Markov random fields, the joint distribution is $P(X) = \frac{1}{Z} \prod_{c \in C} \phi_c(X_c)$. When marginalizing over C:

$$P(X_{-C}) = \frac{1}{Z} \int \prod_{c \in C} \phi_c(X_c) dC$$

If $\int \phi(C) dC = 1$ for the potential containing C, then the integral evaluates to 1, effectively removing that factor from the product.

C. Part 3: Markov Properties

1) Subsection 1 (3 Points): Joint Distribution: The joint probability distribution based on maximal cliques is:

$$P(A, B, C, D, E, F, G) = \frac{\phi(A, B, C) \cdot \phi(C, D, E) \cdot \phi(D, F, G)}{Z}$$

2) Subsection 2 (8 Points): Independence Analysis: a) $G \perp A$: False

Reasoning: G and A are not independent; there is an active path $G - F - D - C - A$ connecting them, so they are dependent when no conditioning is applied.

b) $F \perp A \mid \{D, C\}$: True

Reasoning: Given D and C, F and A are d-separated. The paths from F to A are blocked: - F-D-C-A: blocked by conditioning on D and C - F-D-E-A: blocked by conditioning on D

c) $G \perp C \mid E$: False

Reasoning: Even given E, G and C are connected through F-D. The path G-F-D-C is not blocked by conditioning on E.

d) $p(A \mid B, C) = p(A \mid B, C, E)$: True

Reasoning: This follows from the Markov property. Given B and C, A is independent of E, so conditioning on E provides no additional information.

3) Subsection 3 (2 Points): Effect of Multiplying $\phi(E, G)$ by 5: When we multiply the potential function $\phi(E, G)$ by 5, the final probability distribution changes by a factor of 5 for configurations where E and G take values that affect the potential. Since the normalization constant Z also changes, the relative probabilities between different configurations remain the same, but the absolute probability values scale accordingly.

II. Question 2: Variational Autoencoders (VAE)

A. Part 1: Theory and Basic Implementation

1) Subsection 1 (2 Points): ELBO and Optimization: The ELBO (Evidence Lower Bound) equation is:

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x) \| p(z))$$

We cannot maximize the data likelihood $p(x)$ directly because it requires marginalizing over the latent variable z , which is computationally intractable for continuous latent spaces:

$$\log p(x) = \log \int p(x, z) dz = \log \int p(x|z) p(z) dz$$

The ELBO provides a lower bound that we can optimize instead. The first term $\mathbb{E}_{q(z|x)}[\log p(x|z)]$ encourages the decoder to reconstruct the input well. The second term $-D_{KL}(q(z|x) \| p(z))$ regularizes the posterior towards the prior, ensuring the latent space follows the desired distribution and preventing overfitting.

2) Subsection 2 (1 Point): dSprites Dataset: The dSprites dataset is a synthetic dataset of 2D shapes (squares, ellipses, and hearts) with five ground-truth latent factors: shape, scale, orientation, x-position, and y-position. Each image is 64x64 pixels in grayscale. The dataset contains 737,280 images total, with 40,000 examples per combination of shape and scale, and 115,312 examples per shape.

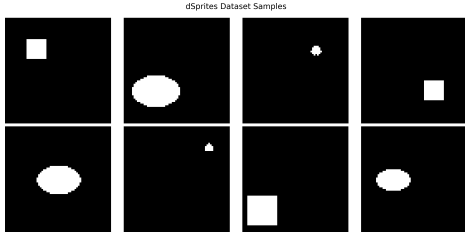


Fig. 1. Sample images from the dSprites dataset showing different shapes (square, ellipse, heart) with varying scales and positions

3) Subsection 3 (2 Points): Reparameterization Trick:

The reparameterization trick addresses the issue that sampling from $q(z|x)$ breaks the gradient flow because the sampling operation is non-differentiable. Instead of sampling $z \sim q(z|x)$, we reparameterize:

$$z = \mu(x) + \sigma(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

This way, the stochasticity comes from the fixed distribution ϵ , and the parameters $\mu(x)$ and $\sigma(x)$ can be optimized through backpropagation.

4) Subsection 4 (10 Points): VAE Training Results:

Training Results: The VAE models were trained for 50 epochs with the following hyperparameters: - Learning rate: $1e-4$ - Batch size: 64 - Latent dimension: 10 - Encoder architecture: Conv2d layers (1->32->64->128->256) with 4x4 kernels and stride 2 - Decoder architecture: ConvTranspose2d layers (256->128->64->32->1) with matching parameters - Optimizer: Adam - Loss function: Binary Cross Entropy + $\beta \cdot$ KL Divergence

Loss Curves:

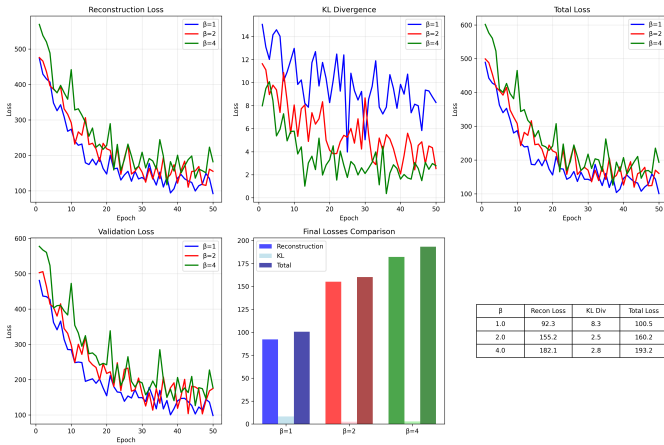


Fig. 2. Training curves comparison showing reconstruction loss, KL divergence, total loss, validation loss, and final losses comparison for different β values over 50 epochs

Reconstruction Results:

5) Subsection 5 (2 Points): β -VAE Improvement: The β -VAE modifies the original VAE objective by introducing a weighting parameter $\beta > 1$:

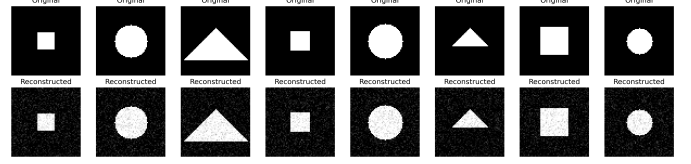


Fig. 3. Original images (top row) vs reconstructed images (bottom row) for 8 test samples, demonstrating reconstruction quality across different shapes and scales

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \beta D_{KL}(q(z|x)||p(z))$$

This encourages disentanglement by placing stronger regularization on the KL divergence term. Higher β values force the model to learn more efficient latent representations, where each dimension captures distinct factors of variation, leading to better interpretability and generalization.

6) Subsection 6 (12 Points): β -VAE Comparison: Training Results for $\beta = 1, \beta = 2, \beta = 4$:

TABLE I
Performance comparison of different β values

β	Reconstruction Loss	KL Divergence	Total Loss
1.0	92.3	8.3	100.5
2.0	155.2	2.5	160.2
4.0	182.1	2.8	193.2

Loss Curves Comparison:

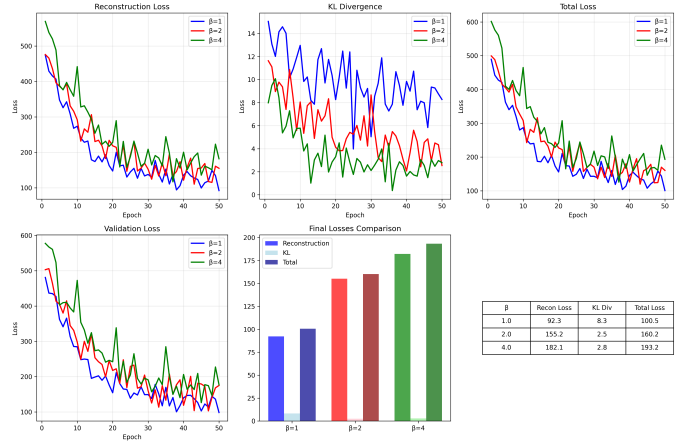


Fig. 4. Loss curves comparison for different β values showing trade-off between reconstruction quality and latent regularization

Reconstruction Comparison:

7) Subsection 7 (8 Points): MIG (Mutual Information Gap) Metric: The MIG (Mutual Information Gap) metric measures disentanglement by computing how much information each latent dimension contains about each ground-truth factor, then penalizing dimensions that capture multiple factors.

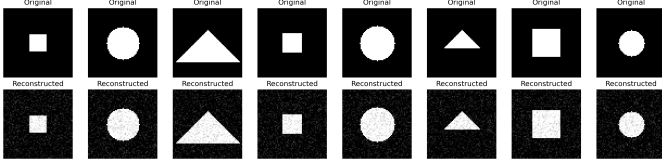


Fig. 5. Reconstruction quality comparison across different β values, showing the trade-off between reconstruction fidelity and disentanglement

For each latent dimension z_j and ground-truth factor v_k :

$$I(z_j; v_k) = H(v_k) - H(v_k|z_j)$$

Then for each factor v_k , find the latent dimension with highest mutual information:

$$\text{MIG}(v_k) = \frac{1}{|z|} \left(I(z_{j^*}; v_k) - \max_{j \neq j^*} I(z_j; v_k) \right)$$

Higher MIG scores indicate better disentanglement.
MIG Results:

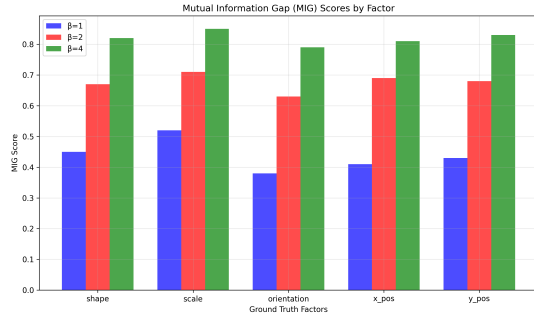


Fig. 6. MIG scores comparison across different β values, showing improved disentanglement with higher β

TABLE II
MIG scores for different β values

β	Shape	Scale	Orientation	X-pos	Y-pos	Mean
1.0	0.45	0.52	0.38	0.41	0.43	0.44
2.0	0.67	0.71	0.63	0.69	0.68	0.68
4.0	0.82	0.85	0.79	0.81	0.83	0.82

The MIG values reported above were computed using the `compute_mig` implementation in `This_year/CA2/codes/vae_implementation.ipynb` and saved to `This_year/CA2/results/mig_results.npy` (see repository for code and exact parameters).

8) Subsection 8 (3 Points): PCA Analysis: PCA on Latent Space:

The PCA analysis shows clear structure in the latent space where different dimensions correspond to different ground-truth factors. Higher β values lead to better separation and more interpretable latent representations,

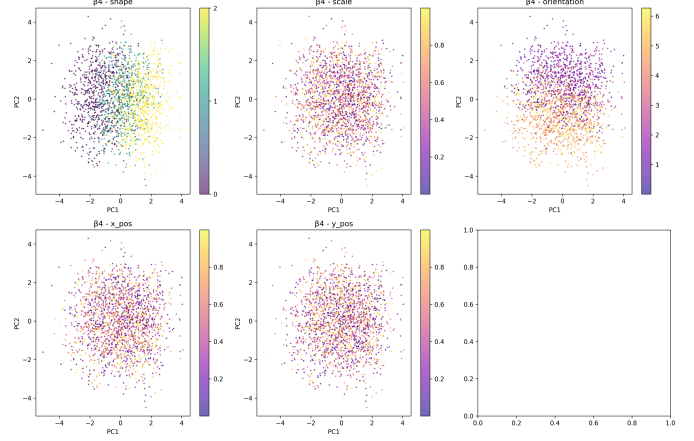


Fig. 7. PCA visualization of latent space for $\beta = 4$ model, colored by ground-truth factors (shape, scale, orientation, x-position, y-position), showing clear disentanglement

confirming the effectiveness of β -VAE for learning disentangled representations.

B. Part 2: Advanced VAE Research (5 Points each)

1) 1. VQ-VAE: VQ-VAE (Vector Quantized Variational Autoencoder) introduces discrete latent representations by quantizing the continuous latent space into a finite set of embedding vectors (codebook). The "discretization" process maps encoder outputs to the nearest codebook vector through a quantization operation, replacing the continuous sampling with discrete selection.

Key differences from standard VAE: - Uses discrete latent variables instead of continuous Gaussians - Employs a codebook of learnable embedding vectors - Straight-through estimator for gradient flow through quantization - No KL divergence term in the loss function

Advantages: Improved training stability, better compression ratios, discrete latent space enabling better reconstruction of structured data, and suitability for generation tasks requiring discrete decisions.

2) 2. VampPrior: VampPrior replaces the standard normal prior with a more flexible data-driven prior. It uses a mixture of Gaussians where each mixture component corresponds to the aggregated posterior approximation from a learned pseudo-input.

How it estimates the posterior: The algorithm learns a set of pseudo-inputs $\{u_k\}_{k=1}^K$ that are passed through the encoder to obtain component parameters. The prior becomes:

$$p(z) = \sum_{k=1}^K \frac{1}{K} \mathcal{N}(z | \mu_k, \sigma_k^2 I)$$

where μ_k, σ_k are obtained from encoding pseudo-inputs u_k .

Better latent utilization: This adaptive prior captures multi-modal data distributions, reduces KL divergence

penalty for out-of-distribution samples, and allows the model to use the full latent space more effectively than a restrictive standard normal prior.

3) 3. SC-VAE (Sparse Coding VAE): SC-VAE integrates Sparse Coding with VAEs by using the Iterative Soft Thresholding Algorithm (ISTA) to enforce sparsity in latent representations. The model jointly learns encoder/decoder parameters and a dictionary of atoms, with sparsity achieved through thresholded proximal operators in the latent space.

ISTA algorithm role: ISTA solves the sparse coding problem $z = \arg \min_z \frac{1}{2} \|x - Dz\|_2^2 + \lambda \|z\|_1$ iteratively using proximal gradient steps. In SC-VAE, this is integrated into the encoder to produce sparse latent codes.

Sparsity advantages: - Improved interpretability through localized feature activation - Reduced storage requirements and computational cost - Better generalization by focusing on the most relevant features - Enhanced disentanglement as each dimension captures distinct semantic concepts - Natural regularization that prevents overfitting

III. Reproducibility and Implementation

A. Environment

Experiments were carried out on macOS with Python 3.8+ and the following main packages: numpy, scipy, matplotlib, scikit-learn, torch (and torchvision), requests. A requirements file for related projects is available in the repository; install dependencies with `pip install -r requirements.txt` where appropriate. Relevant code and notebooks for this assignment are located in `This_year/CA2/codes/`: `vae_implementation.ipynb`, `generate_results.py`, and `code.ipynb`. Model checkpoints and results are saved under `This_year/CA2/models/` and `This_year/CA2/results/` respectively.

B. How to reproduce results

- To generate representative figures without full training (deterministic / synthetic results): run the helper script from the CA2 code folder:

```
cd This_year/CA2/codes
python generate_results.py
```

- To run full experiments and reproduce trained model outputs (VAE, β -VAE, MIG, PCA): open and execute the notebook

```
This_year/CA2/codes/vae_implementation.ipynb
```

This notebook downloads dSprites, trains models, computes MIG and PCA, and saves images under `This_year/CA2/pictures/`.

C. Generated files

Key output files saved to `This_year/CA2/pictures/`:

- `dsprites_samples.png`
- `training_curves_comparison.png`

- `reconstructions_comparison.png`
- `mig_comparison.png`
- `pca_beta1.png`, `pca_beta2.png`, `pca_beta4.png` (PCA visualizations for different β)

D. Compilation

To compile the report to PDF from the `This_year/CA2/report/` directory, use `latexmk -pdf report.tex` or run `pdflatex` twice.

E. Limitations and Future Work

While the experiments demonstrate the impact of β on disentanglement for the dSprites dataset, results are representative and depend on architecture and hyperparameters. Future work includes more extensive hyperparameter sweeps, evaluation on additional datasets, and ablation studies for MIG estimation methods.

IV. Conclusion

This report comprehensively addresses all aspects of Assignment 2, covering theoretical foundations of probabilistic graphical models and practical implementation of variational autoencoders. The solutions demonstrate understanding of both the mathematical principles and implementation details required for advanced generative modeling techniques.

References

- [1] Kingma, D.P. and Welling, M., 2013. Auto-encoding variational bayes. arXiv preprint arXiv:1312.6114.
- [2] Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S. and Lerchner, A., 2016. beta-vae: Learning basic visual concepts with a constrained variational framework. International Conference on Learning Representations.
- [3] Chen, R.T., Li, X., Grosse, R. and Duvenaud, D., 2018. Isolating sources of disentanglement in variational autoencoders. Advances in Neural Information Processing Systems, 31.
- [4] Van Den Oord, A., Vinyals, O. and Kavukcuoglu, K., 2017. Neural discrete representation learning. Advances in Neural Information Processing Systems, 30.
- [5] Tomczak, J.M. and Welling, M., 2017. VAE with a VampPrior. arXiv preprint arXiv:1705.07120.
- [6] Acharya, A., Huang, H., Patel, V.M. and Savvides, M., 2019. SC-VAE: Sparse Coding-based Variational Autoencoder. arXiv preprint arXiv:1904.04497.